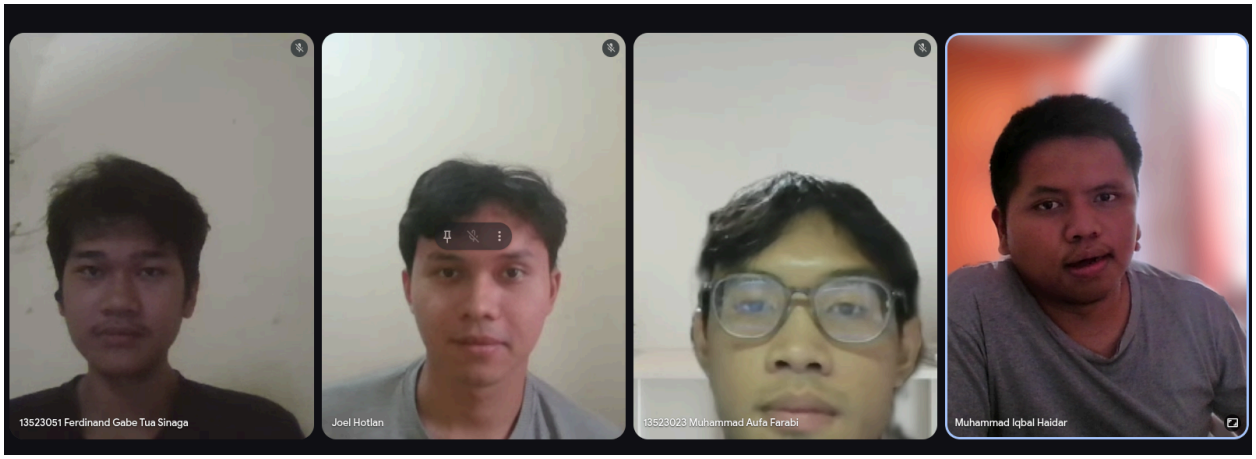
IF2224 TEORI BAHASA FORMAL DAN OTOMATA
SEMESTER II TAHUN 2025/2026

PASCAL-S COMPILER



Milestone 2 - Syntax Analysis

Kelompok LPH

Muhammad Aufa Farabi	13523023
Joel Hotlan Haris Siahaan	13523025
Ferdinand Gabe Tua Sinaga	13523051
Muhammad Iqbal Haidar	13523111

PROGRAM STUDI TEKNIK INFORMATIKA
SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA
INSTITUT TEKNOLOGI BANDUNG
2025

DAFTAR ISI

LANDASAN TEORI

Pada proses kompilasi bahasa pemrograman, tahapan setelah analisis leksikal adalah analisis sintaks. Pada tahap ini, parser akan memastikan jika urutan token hasil dari analisis leksikal sesuai dengan grammar dari bahasa pemrograman yang digunakan. Struktur data parse tree digunakan untuk merepresentasikan struktur hierarkis antara token dan produksi grammar.

Parser dibangun dengan metode top-down dalam menurunkan simbol-simbol grammar. Jika rangkaian token tidak sesuai dengan aturan grammar, maka parser akan menghasilkan syntax error dan proses kompilasi dihentikan. Dalam algoritma recursive descent parser setiap aturan grammar (simbol non-terminal) diimplementasikan sebagai sebuah fungsi terpisah. Fungsi-fungsi ini saling memanggil secara rekursif untuk menelusuri aturan. Parser akan melihat token berikutnya (lookahead) untuk menentukan jalur produksi mana yang harus diambil dan mencocokkan token terminal (tanpa backtracking). Analisis sintaks merupakan tahap penting untuk memastikan struktur program sudah benar sebelum dilanjutkan ke analisis semantik.

PERANCANGAN & IMPLEMENTASI

Untuk membangun parser pada milestone kali ini, kelompok kami memutuskan untuk membuat dua kelas utama, yaitu kelas *TreeNode* dan kelas *Parser*. Di dalam kelas *Parser*, *TreeNode* dimanfaatkan sebagai representasi node pada *parse tree*.

Implementasi: Class TreeNode	
Fungsi	Keterangan
add_child(node: TreeNode)	Fungsi ini digunakan untuk menambahkan anak ke suatu node tertentu. Fungsi ini merupakan fungsi utama yang digunakan untuk merepresentasikan *parse tree*, mulai dari variabel hingga terminal state.
display()	Fungsi ini digunakan untuk menampilkan *parse tree* kepada pengguna.
_display_children_recursive(indent: int)	Fungsi ini digunakan untuk membangun representasi *parse tree* dalam bentuk string secara rekursif, sehingga memudahkan pengguna dalam membaca struktur tree yang dihasilkan.

Kelas *Parser* merupakan kelas utama yang melakukan pengecekan sintaks terhadap program berekstensi .pas. Secara garis besar, kelas ini bertanggung jawab untuk menentukan apakah sebuah string valid atau tidak berdasarkan *production rule* yang ada. Jika valid, kelas ini akan menambahkan node anak ke parent-nya. Jika tidak valid, kelas ini akan menampilkan pesan kesalahan yang menjelaskan jenis dan lokasi kesalahan sintaks pada program tersebut.

Implementasi: Class Parser	
Nama Fungsi	Keterangan
read()	Fungsi ini digunakan untuk membaca satu token pada setiap pemanggilannya. Setiap kali fungsi ini dipanggil, nilai idx_pointer akan digeser sebesar satu (idx_pointer += 1).
accept(type , value =None) -> TreeNode	Fungsi ini dimanfaatkan untuk melakukan pengecekan apakah suatu token diterima pada suatu production rule tertentu jika diterima maka dia akan mengembalikan sebuah node dan jika tidak akan menampilkan pesan error dan exit dari program
peek()	Fungsi ini dibuat untuk melakukan *look-ahead* pada token, yang berguna untuk mengatasi ambiguitas grammar, khususnya pada bagian *statement*, seperti membedakan apakah sebuah *identifier* merupakan awal dari *assignment statement* atau *procedure/function call*.
parse()	Fungsi ini berguna untuk memulai pembangunan dari *parse tree.*
program() -> TreeNode	Fungsi ini merupakan hasil translasi production rule <program> ::= <program-header> <declaration-part> <compound-statement> DOT(.) Mengembalikan object TreeNode dengan atribut nama = <program>
program_header() -> TreeNode	Fungsi ini merupakan hasil translasi production

	rule <program_header> ::= KEYWORD(program) IDENTIFIER SEMICOLON(;). Mengembalikan object TreeNode dengan atribut nama = <program-header>
declaration_part() -> TreeNode	Fungsi ini merupakan hasil translasi production rule <declaration_part> ::= {const-declaration} {type-declaration} {var-declaration} {subprogram-declaration}. Mengembalikan object TreeNode dengan atribut nama = <declaration-part>
const_declaration() -> TreeNode	Fungsi ini melakukan parsing konstanta dengan berbagai nilai seperti angka, karakter, dan string.
type_declaration() -> TreeNode	Fungsi ini memparsing berbagai jenis deklarasi tipe data, mulai dari tipe dasar hingga tipe terstruktur seperti larik dan rekaman.
record_type() -> TreeNode	Fungsi ini melakukan parsing khusus untuk tipe data yang berbentuk rekaman
var_declaration() -> TreeNode	Fungsi ini merupakan hasil translasi production rule <var_declaration> ::= KEYWORD(variabel) (<identifier-list> COLON(:) type SEMICOLON(;)) {<identifier-list> COLON(:) type SEMICOLON(;)}. Mengembalikan object TreeNode dengan atribut nama = <var-declaration>
identifier_list() -> TreeNode	Fungsi ini merupakan hasil translasi production rule <identifier_list> ::= IDENTIFIER {COMMA(,) IDENTIFIER}. Mengembalikan object TreeNode dengan atribut nama = <identifier-list>
type() -> TreeNode	Fungsi yang mengenali dan memparsing tipe data, baik tipe standar seperti integer, real, char, boolean, atau user defined type dalam deklarasi variabel yang mengembalikan internal node parse tree tersebut
array_type() -> TreeNode	Fungsi ini berguna untuk melakukan parsing khusus untuk tipe data berbentuk larik
range() -> TreeNode	Fungsi ini merupakan hasil translasi production rule <range> ::= <expression> RANGE_OPERATOR(..) <expression>. Mengembalikan object TreeNode dengan atribut nama = <range>

subprogram_declaration() -> TreeNode	Fungsi ini melakukan parsing deklarasi fungsi atau prosedur beserta parameternya yang bersifat opsional.
procedure_declaration() -> TreeNode	Fungsi ini merupakan hasil translasi production rule <procedure-declaration> ::= KEYWORD(prosedur) IDENTIFIER [<formal-parameter-list>] SEMICOLON(;) <block> SEMICOLON(;). Mengembalikan object TreeNode dengan atribut nama = <procedure-declaration>
function_declaration() -> TreeNode	Fungsi yang memparsing deklarasi fungsi berdasarkan struktur deklarasi fungsi sesuai grammar nya (diawali keyword(Fungsi) dan diakhiri semicolon)
formal_parameter_list() -> TreeNode	Fungsi ini melakukan parsing untuk deklarasi daftar parameter formal pada header suatu fungsi atau prosedur beserta tipenya.
parameter_group() -> TreeNode	Fungsi ini melakukan parsing untuk sebuah grup parameter yang berisi nama-nama parameter terpisahkan oleh COLON beserta tipe parameter.
compound_statement() -> TreeNode	Fungsi ini merupakan hasil translasi production rule <compound_statement> ::= KEYWORD(mulai) <statement-list> KEYWORD(selesai). Mengembalikan object TreeNode dengan atribut nama = <compound-statement>
statement_list() -> TreeNode	Fungsi ini melakukan parsing untuk daftar pernyataan pada sebuah blok, yang setiap pernyataannya dipisahkan oleh SEMICOLON.
assignment_statement() -> TreeNode	Fungsi ini melakukan parsing pernyataan assignment yaitu pemberian nilai ke suatu variabel.
if_statement() -> TreeNode	Fungsi ini berguna untuk parsing struktur percabangan jika … maka … serta bagian opsional selainitu
while_statement() -> TreeNode	fungsi yang memparsing struktur perulangan selama … lakukan …
for_statement() -> TreeNode	Fungsi ini merupakan hasil translasi production rule <for_statement> ::= KEYWORD(untuk) IDENTIFIER ASSIGN_OPERATOR(:=) <expression> (KEYWORD(ke) |

	KEYWORD(turun-ke)) <expression> KEYWORD(lakukan) <statement>. Mengembalikan object TreeNode dengan atribut nama = <for-statement>
procedure_function_call() -> TreeNode	Au Fungsi yang memparsing pemanggilan prosedur atau fungsi berdasarkan struktur identifier nama prosedur atau fungsi + (list parameter) nya
parameter_list() -> TreeNode	Fungsi ini berguna untuk parsing daftar parameter dari sebuah pemanggilan fungsi.
expression() -> TreeNode	Fungsi yang mengenali dan memparsing suatu ekspresi yang menghasilkan nilai
simple_expression() -> TreeNode	Fungsi ini melakukan parsing untuk ekspresi sederhana yaitu ekspresi yang tidak mengandung operator relasional.
term() -> TreeNode	Fungsi ini merupakan hasil translasi production rule <term> ::= <factor> {<multiplicative-operator> <factor>}. Mengembalikan object TreeNode dengan atribut nama = <term>
factor() -> TreeNode	Fungsi yang memparsing unit-unit terkecil dari ekspresi, seperti char, number, ekspresi dalam parenthesis, atau dirinya sendiri dengan kondisi “tidak”(logic operator).
variable() -> TreeNode	Fungsi yang melakukan parsing terhadap variabel, berupa variabel simple, misalnya “x”, variabel atribut dari record, misalnya “rec.x”, atau variabel array, misalnya “arr[1]”
relational_operator() -> TreeNode	Fungsi yang melakukan parsing terhadap operator perbandingan (=, <>, <, <=, >, >=)
additive_operator() -> TreeNode	Fungsi ini digunakan untuk melakukan parsing terhadap operator aritmetika tingkat penjumlahan seperti + dan -, maupun operator logika atau
multiplicative_operator() -> TreeNode	Fungsi ini dibuat dengan tujuan melakukan parsing terhadap operator aritmetika tingkat perkalian seperti *, /, bagi, mod maupun operator logika dan
statement() -> TreeNode	Fungsi ini melakukan parsing untuk semua jenis statement dengan memanggil parser setiap statement yang dibaca.

case_statement() -> TreeNode	Fungsi ini memparsing statement kasus beserta ekspresi penguji, daftar konstanta, dan statement untuk setiap cabang hingga kata kunci akhir
const() -> TreeNode	Fungsi ini Memparsing nilai konstan seperti angka, karakter, identifier, atau angka dengan unary +/-
repeat_statement() -> TreeNode	Fungsi ini merupakan hasil translasi production rule <repeat_statement> ::= KEYWORD(ulangi) <statement-list> KEYWORD(sampai) <expression>. Mengembalikan object TreeNode dengan atribut nama = <repeat-statement>

PENGUJIAN

Kasus 1

Input file: test1-3.pas

```
program RekamanTest;

tipe
  Orang = rekaman
    nama : char;
    umur : integer;
    tinggi, berat : integer;
  selesai;

variabel
  o : Orang;

mulai
  o.umur := 20;
selesai.
```

Output

```
Parse success, code error free
<program>
|- <program-header>
|  |- KEYWORD(program)
|  |- IDENTIFIER(RekamanTest)
|  L- SEMICOLON(;)
|- <declaration-part>
|  |- <type-declaration>
|  |  |- KEYWORD(tipe)
```

```
|   |   |- IDENTIFIER(Orang)
|   |   |- RELATIONAL_OPERATOR(=)
|   |   |- <type>
|   |   |   L- <record-type>
|   |   |      |- KEYWORD(rekaman)
|   |   |      |- <identifier-list>
|   |   |      |   L- IDENTIFIER(nama)
|   |   |      |- COLON(:)
|   |   |      |- <type>
|   |   |      |   L- KEYWORD(char)
|   |   |      |- SEMICOLON(;)
|   |   |      |- <identifier-list>
|   |   |      |   L- IDENTIFIER(umur)
|   |   |      |- COLON(:)
|   |   |      |- <type>
|   |   |      |   L- KEYWORD(integer)
|   |   |      |- SEMICOLON(;)
|   |   |      |- <identifier-list>
|   |   |      |   |- IDENTIFIER(tinggi)
|   |   |      |   |- COMMA(,)
|   |   |      |   L- IDENTIFIER(berat)
|   |   |      |- COLON(:)
|   |   |      |- <type>
|   |   |      |   L- KEYWORD(integer)
|   |   |      |- SEMICOLON(;)
|   |   |      L- KEYWORD(selesai)
|   |   L- SEMICOLON(;)
|   L- <var-declaration>
|      |- KEYWORD(variabel)
|      |- <identifier-list>
|      |   L- IDENTIFIER(o)
|      |- COLON(:)
|      |- <type>
|      |   L- IDENTIFIER(Orang)
|      L- SEMICOLON(;)
|- <compound-statement>
|   |- KEYWORD(mulai)
|   |- <statement-list>
|   |   |- <statement>
|   |   |   L- <assignment-statement>
|   |   |      |- <variable>
|   |   |      |   |- IDENTIFIER(o)
|   |   |      |   |- DOT(.)
|   |   |      |   L- IDENTIFIER(umur)
|   |   |      |- ASSIGN_OPERATOR(:=)
|   |   |      L- <expression>
|   |   |         L- <simple-expression>
|   |   |            L- <term>
|   |   |               L- <factor>
|   |   |                  L- NUMBER(20)
|   |   |- SEMICOLON(;)
|   |   L- <statement>
|   L- KEYWORD(selesai)
L- DOT(.)
```

Kasus 2

Input file: test1-10.pas

```
program UjiSemua;

konstanta
  max = 10;
  angka = 5;

tipe
  Angka = integer;
  Data = rekaman
    x, y : integer;
    z : char;
  selesai;
  Matriks = larik[1 .. 5] dari Angka;

variabel
  a, b : integer;
  d : Data;
  m : Matriks;

mulai
  a := 1;
  b := max;

  d.x := 10;
  d.y := 20;
  d.z := 'A';

  m[1] := 100;

selesai.
```

Output

```
Parse success, code error free
<program>
|- <program-header>
|  |- KEYWORD(program)
|  |- IDENTIFIER(UjiSemua)
|  L- SEMICOLON(;)
|- <declaration-part>
|  |- <const-declaration>
|  |  |- KEYWORD(konstanta)
|  |  |- IDENTIFIER(max)
|  |  |- RELATIONAL_OPERATOR(=)
|  |  |- <const>
|  |  |  L- <number-statement>
|  |  |     L- NUMBER(10)
|  |  |- SEMICOLON(;)
|  |  |- IDENTIFIER(angka)
|  |  |- RELATIONAL_OPERATOR(=)
|  |  |- <const>
|  |  |  L- <number-statement>
```

```
|  |  |     L- NUMBER(5)
|  |  L- SEMICOLON(;)
|  |- <type-declaration>
|  |  |- KEYWORD(tipe)
|  |  |- IDENTIFIER(Angka)
|  |  |- RELATIONAL_OPERATOR(=)
|  |  |- <type>
|  |  |  L- KEYWORD(integer)
|  |  |- SEMICOLON(;)
|  |  |- IDENTIFIER(Data)
|  |  |- RELATIONAL_OPERATOR(=)
|  |  |- <type>
|  |  |  L- <record-type>
|  |  |     |- KEYWORD(rekaman)
|  |  |     |- <identifier-list>
|  |  |     |  |- IDENTIFIER(x)
|  |  |     |  |- COMMA(,)
|  |  |     |  L- IDENTIFIER(y)
|  |  |     |- COLON(:)
|  |  |     |- <type>
|  |  |     |  L- KEYWORD(integer)
|  |  |     |- SEMICOLON(;)
|  |  |     |- <identifier-list>
|  |  |     |  L- IDENTIFIER(z)
|  |  |     |- COLON(:)
|  |  |     |- <type>
|  |  |     |  L- KEYWORD(char)
|  |  |     |- SEMICOLON(;)
|  |  |     L- KEYWORD(selesai)
|  |  |- SEMICOLON(;)
|  |  |- IDENTIFIER(Matriks)
|  |  |- RELATIONAL_OPERATOR(=)
|  |  |- <type>
|  |  |  L- <array-type>
|  |  |     |- KEYWORD(larik)
|  |  |     |- LBRACKET([)
|  |  |     |- <range>
|  |  |     |  |- <expression>
|  |  |     |  |  L- <simple-expression>
|  |  |     |  |     L- <term>
|  |  |     |  |        L- <factor>
|  |  |     |  |           L- NUMBER(1)
|  |  |     |  |- RANGE_OPERATOR(..)
|  |  |     |  L- <expression>
|  |  |     |     L- <simple-expression>
|  |  |     |        L- <term>
|  |  |     |           L- <factor>
|  |  |     |              L- NUMBER(5)
|  |  |     |- RBRACKET(])
|  |  |     |- KEYWORD(dari)
|  |  |     L- <type>
|  |  |        L- IDENTIFIER(Angka)
|  |  L- SEMICOLON(;)
|  L- <var-declaration>
|     |- KEYWORD(variabel)
|     |- <identifier-list>
```

```
|     |  |- IDENTIFIER(a)
|     |  |- COMMA(,)
|     |  L- IDENTIFIER(b)
|     |- COLON(:)
|     |- <type>
|     |  L- KEYWORD(integer)
|     |- SEMICOLON(;)
|     |- <identifier-list>
|     |  L- IDENTIFIER(d)
|     |- COLON(:)
|     |- <type>
|     |  L- IDENTIFIER(Data)
|     |- SEMICOLON(;)
|     |- <identifier-list>
|     |  L- IDENTIFIER(m)
|     |- COLON(:)
|     |- <type>
|     |  L- IDENTIFIER(Matriks)
|     L- SEMICOLON(;)
|- <compound-statement>
|  |- KEYWORD(mulai)
|  |- <statement-list>
|  |  |- <statement>
|  |  |  L- <assignment-statement>
|  |  |     |- <variable>
|  |  |     |  L- IDENTIFIER(a)
|  |  |     |- ASSIGN_OPERATOR(:=)
|  |  |     L- <expression>
|  |  |        L- <simple-expression>
|  |  |           L- <term>
|  |  |              L- <factor>
|  |  |                 L- NUMBER(1)
|  |  |- SEMICOLON(;)
|  |  |- <statement>
|  |  |  L- <assignment-statement>
|  |  |     |- <variable>
|  |  |     |  L- IDENTIFIER(b)
|  |  |     |- ASSIGN_OPERATOR(:=)
|  |  |     L- <expression>
|  |  |        L- <simple-expression>
|  |  |           L- <term>
|  |  |              L- <factor>
|  |  |                 L- <variable>
|  |  |                    L- IDENTIFIER(max)
|  |  |- SEMICOLON(;)
|  |  |- <statement>
|  |  |  L- <assignment-statement>
|  |  |     |- <variable>
|  |  |     |  |- IDENTIFIER(d)
|  |  |     |  |- DOT(.)
|  |  |     |  L- IDENTIFIER(x)
|  |  |     |- ASSIGN_OPERATOR(:=)
|  |  |     L- <expression>
|  |  |        L- <simple-expression>
|  |  |           L- <term>
|  |  |              L- <factor>
```

```
|  |  |                 L- NUMBER(10)
|  |  |- SEMICOLON(;)
|  |  |- <statement>
|  |  |  L- <assignment-statement>
|  |  |     |- <variable>
|  |  |     |  |- IDENTIFIER(d)
|  |  |     |  |- DOT(.)
|  |  |     |  L- IDENTIFIER(y)
|  |  |     |- ASSIGN_OPERATOR(:=)
|  |  |     L- <expression>
|  |  |        L- <simple-expression>
|  |  |           L- <term>
|  |  |              L- <factor>
|  |  |                 L- NUMBER(20)
|  |  |- SEMICOLON(;)
|  |  |- <statement>
|  |  |  L- <assignment-statement>
|  |  |     |- <variable>
|  |  |     |  |- IDENTIFIER(d)
|  |  |     |  |- DOT(.)
|  |  |     |  L- IDENTIFIER(z)
|  |  |     |- ASSIGN_OPERATOR(:=)
|  |  |     L- <expression>
|  |  |        L- <simple-expression>
|  |  |           L- <term>
|  |  |              L- <factor>
|  |  |                 L- CHAR_LITERAL('A')
|  |  |- SEMICOLON(;)
|  |  |- <statement>
|  |  |  L- <assignment-statement>
|  |  |     |- <variable>
|  |  |     |  |- IDENTIFIER(m)
|  |  |     |  |- LBRACKET([)
|  |  |     |  |- <parameter-list>
|  |  |     |  |  L- <expression>
|  |  |     |  |     L- <simple-expression>
|  |  |     |  |        L- <term>
|  |  |     |  |           L- <factor>
|  |  |     |  |              L- NUMBER(1)
|  |  |     |  L- RBRACKET(])
|  |  |     |- ASSIGN_OPERATOR(:=)
|  |  |     L- <expression>
|  |  |        L- <simple-expression>
|  |  |           L- <term>
|  |  |              L- <factor>
|  |  |                 L- NUMBER(100)
|  |  |- SEMICOLON(;)
|  |  L- <statement>
|  L- KEYWORD(selesai)
L- DOT(.)
```

Kasus 3

Input file: test1-alldeclare.pas

```
program MiniTest;

konstanta
  k := 9;

tipe
  Bil = integer;
  Mahasiswa = rekaman
    nama : char;
    umur : integer;
  selesai;

variabel
  x : Bil;
  mhs : Mahasiswa;

mulai
  x := k;
selesai.
```

Output

```
Parse success, code error free
<program>
|- <program-header>
|  |- KEYWORD(program)
|  |- IDENTIFIER(MiniTest)
|  L- SEMICOLON(;)
|- <declaration-part>
|  |- <const-declaration>
|  |  |- KEYWORD(konstanta)
|  |  |- IDENTIFIER(k)
|  |  |- RELATIONAL_OPERATOR(=)
|  |  |- <const>
|  |  |  L- <number-statement>
|  |  |     L- NUMBER(9)
|  |  L- SEMICOLON(;)
|  |- <type-declaration>
|  |  |- KEYWORD(tipe)
|  |  |- IDENTIFIER(Bil)
|  |  |- RELATIONAL_OPERATOR(=)
|  |  |- <type>
|  |  |  L- KEYWORD(integer)
|  |  |- SEMICOLON(;)
|  |  |- IDENTIFIER(Mahasiswa)
|  |  |- RELATIONAL_OPERATOR(=)
|  |  |- <type>
|  |  |  L- <record-type>
|  |  |     |- KEYWORD(rekaman)
|  |  |     |- <identifier-list>
|  |  |     |  L- IDENTIFIER(nama)
|  |  |     |- COLON(:)
|  |  |     |- <type>
|  |  |     |  L- KEYWORD(char)
|  |  |     |- SEMICOLON(;)
```

```
|  |  |     |- <identifier-list>
|  |  |     |  L- IDENTIFIER(umur)
|  |  |     |- COLON(:)
|  |  |     |- <type>
|  |  |     |  L- KEYWORD(integer)
|  |  |     |- SEMICOLON(;)
|  |  |     L- KEYWORD(selesai)
|  |  L- SEMICOLON(;)
|  L- <var-declaration>
|     |- KEYWORD(variabel)
|     |- <identifier-list>
|     |  L- IDENTIFIER(x)
|     |- COLON(:)
|     |- <type>
|     |  L- IDENTIFIER(Bil)
|     |- SEMICOLON(;)
|     |- <identifier-list>
|     |  L- IDENTIFIER(mhs)
|     |- COLON(:)
|     |- <type>
|     |  L- IDENTIFIER(Mahasiswa)
|     L- SEMICOLON(;)
|- <compound-statement>
|  |- KEYWORD(mulai)
|  |- <statement-list>
|  |  |- <statement>
|  |  |  L- <assignment-statement>
|  |  |     |- <variable>
|  |  |     |  L- IDENTIFIER(x)
|  |  |     |- ASSIGN_OPERATOR(:=)
|  |  |     L- <expression>
|  |  |        L- <simple-expression>
|  |  |           L- <term>
|  |  |              L- <factor>
|  |  |                 L- <variable>
|  |  |                    L- IDENTIFIER(k)
|  |  |- SEMICOLON(;)
|  |  L- <statement>
|  L- KEYWORD(selesai)
L- DOT(.)
```

Kasus 4

Input file: test1-const

```
program ConstTest;

konstanta
  satu = 1;
  dua = 2;
  tiga = 3;

variabel
  x : integer;
```

```
mulai
  x := dua;
selesai.
```

Output

```
Parse success, code error free
<program>
|- <program-header>
|  |- KEYWORD(program)
|  |- IDENTIFIER(ConstTest)
|  L- SEMICOLON(;)
|- <declaration-part>
|  |- <const-declaration>
|  |  |- KEYWORD(konstanta)
|  |  |- IDENTIFIER(satu)
|  |  |- RELATIONAL_OPERATOR(=)
|  |  |- <const>
|  |  |  L- <number-statement>
|  |  |     L- NUMBER(1)
|  |  |- SEMICOLON(;)
|  |  |- IDENTIFIER(dua)
|  |  |- RELATIONAL_OPERATOR(=)
|  |  |- <const>
|  |  |  L- <number-statement>
|  |  |     L- NUMBER(2)
|  |  |- SEMICOLON(;)
|  |  |- IDENTIFIER(tiga)
|  |  |- RELATIONAL_OPERATOR(=)
|  |  |- <const>
|  |  |  L- <number-statement>
|  |  |     L- NUMBER(3)
|  |  L- SEMICOLON(;)
|  L- <var-declaration>
|     |- KEYWORD(variabel)
|     |- <identifier-list>
|     |  L- IDENTIFIER(x)
|     |- COLON(:)
|     |- <type>
|     |  L- KEYWORD(integer)
|     L- SEMICOLON(;)
|- <compound-statement>
|  |- KEYWORD(mulai)
|  |- <statement-list>
|  |  |- <statement>
|  |  |  L- <assignment-statement>
|  |  |     |- <variable>
|  |  |     |  L- IDENTIFIER(x)
|  |  |     |- ASSIGN_OPERATOR(:=)
|  |  |     L- <expression>
|  |  |        L- <simple-expression>
|  |  |           L- <term>
|  |  |              L- <factor>
|  |  |                 L- <variable>
|  |  |                    L- IDENTIFIER(dua)
```

```
|  |  |- SEMICOLON(;)
|  |  L- <statement>
|  L- KEYWORD(selesai)
L- DOT(.)
```

Kasus 5

Input file: test1-larik.pas

```
program ArrayTest;

tipe
  Angka5 = larik[1 .. 5] dari integer;

variabel
  a : Angka5;

mulai
  a[1] := 7;
  a[2] := 8;
selesai.
```

Output

```
Parse success, code error free
<program>
|- <program-header>
|  |- KEYWORD(program)
|  |- IDENTIFIER(ArrayTest)
|  L- SEMICOLON(;)
|- <declaration-part>
|  |- <type-declaration>
|  |  |- KEYWORD(tipe)
|  |  |- IDENTIFIER(Angka5)
|  |  |- RELATIONAL_OPERATOR(=)
|  |  |- <type>
|  |  |  L- <array-type>
|  |  |     |- KEYWORD(larik)
|  |  |     |- LBRACKET([)
|  |  |     |- <range>
|  |  |     |  |- <expression>
|  |  |     |  |  L- <simple-expression>
|  |  |     |  |     L- <term>
|  |  |     |  |        L- <factor>
|  |  |     |  |           L- NUMBER(1)
|  |  |     |  |- RANGE_OPERATOR(..)
|  |  |     |  L- <expression>
|  |  |     |     L- <simple-expression>
|  |  |     |        L- <term>
|  |  |     |           L- <factor>
|  |  |     |              L- NUMBER(5)
|  |  |     |- RBRACKET(])
|  |  |     |- KEYWORD(dari)
|  |  |     L- <type>
```

```
|  |  |        L- KEYWORD(integer)
|  |  L- SEMICOLON(;)
|  L- <var-declaration>
|     |- KEYWORD(variabel)
|     |- <identifier-list>
|     |  L- IDENTIFIER(a)
|     |- COLON(:)
|     |- <type>
|     |  L- IDENTIFIER(Angka5)
|     L- SEMICOLON(;)
|- <compound-statement>
|  |- KEYWORD(mulai)
|  |- <statement-list>
|  |  |- <statement>
|  |  |  L- <assignment-statement>
|  |  |     |- <variable>
|  |  |     |  |- IDENTIFIER(a)
|  |  |     |  |- LBRACKET([)
|  |  |     |  |- <parameter-list>
|  |  |     |  |  L- <expression>
|  |  |     |  |     L- <simple-expression>
|  |  |     |  |        L- <term>
|  |  |     |  |           L- <factor>
|  |  |     |  |              L- NUMBER(1)
|  |  |     |  L- RBRACKET(])
|  |  |     |- ASSIGN_OPERATOR(:=)
|  |  |     L- <expression>
|  |  |        L- <simple-expression>
|  |  |           L- <term>
|  |  |              L- <factor>
|  |  |                 L- NUMBER(7)
|  |  |- SEMICOLON(;)
|  |  |- <statement>
|  |  |  L- <assignment-statement>
|  |  |     |- <variable>
|  |  |     |  |- IDENTIFIER(a)
|  |  |     |  |- LBRACKET([)
|  |  |     |  |- <parameter-list>
|  |  |     |  |  L- <expression>
|  |  |     |  |     L- <simple-expression>
|  |  |     |  |        L- <term>
|  |  |     |  |           L- <factor>
|  |  |     |  |              L- NUMBER(2)
|  |  |     |  L- RBRACKET(])
|  |  |     |- ASSIGN_OPERATOR(:=)
|  |  |     L- <expression>
|  |  |        L- <simple-expression>
|  |  |           L- <term>
|  |  |              L- <factor>
|  |  |                 L- NUMBER(8)
|  |  |- SEMICOLON(;)
|  |  L- <statement>
|  L- KEYWORD(selesai)
L- DOT(.)
```

Kasus 6

Input file: test1.pas

```
program test1;

variabel
    x, y, total: integer;

mulai
    x := 10;
    y := 5;
    total := x + y;
selesai.
```

Output

```
Parse success, code error free
<program>
|- <program-header>
|  |- KEYWORD(program)
|  |- IDENTIFIER(test1)
|  L- SEMICOLON(;)
|- <declaration-part>
|  L- <var-declaration>
|     |- KEYWORD(variabel)
|     |- <identifier-list>
|     |  |- IDENTIFIER(x)
|     |  |- COMMA(,)
|     |  |- IDENTIFIER(y)
|     |  |- COMMA(,)
|     |  L- IDENTIFIER(total)
|     |- COLON(:)
|     |- <type>
|     |  L- KEYWORD(integer)
|     L- SEMICOLON(;)
|- <compound-statement>
|  |- KEYWORD(mulai)
|  |- <statement-list>
|  |  |- <statement>
|  |  |  L- <assignment-statement>
|  |  |     |- <variable>
|  |  |     |  L- IDENTIFIER(x)
|  |  |     |- ASSIGN_OPERATOR(:=)
|  |  |     L- <expression>
|  |  |        L- <simple-expression>
|  |  |           L- <term>
|  |  |              L- <factor>
|  |  |                 L- NUMBER(10)
|  |  |- SEMICOLON(;)
|  |  |- <statement>
|  |  |  L- <assignment-statement>
|  |  |     |- <variable>
|  |  |     |  L- IDENTIFIER(y)
|  |  |     |- ASSIGN_OPERATOR(:=)
|  |  |     L- <expression>
```

```
|  |  |        L- <simple-expression>
|  |  |           L- <term>
|  |  |              L- <factor>
|  |  |                 L- NUMBER(5)
|  |  |- SEMICOLON(;)
|  |  |- <statement>
|  |  |  L- <assignment-statement>
|  |  |     |- <variable>
|  |  |     |  L- IDENTIFIER(total)
|  |  |     |- ASSIGN_OPERATOR(:=)
|  |  |     L- <expression>
|  |  |        L- <simple-expression>
|  |  |           |- <term>
|  |  |           |  L- <factor>
|  |  |           |     L- <variable>
|  |  |           |        L- IDENTIFIER(x)
|  |  |           |- ARITHMETIC_OPERATOR(+)
|  |  |           L- <term>
|  |  |              L- <factor>
|  |  |                 L- <variable>
|  |  |                    L- IDENTIFIER(y)
|  |  |- SEMICOLON(;)
|  |  L- <statement>
|  L- KEYWORD(selesai)
L- DOT(.)
```

Kasus 7

Input file: test2.pas

```
program test2;

variabel
    s: string;
    c: char;

mulai
    s := 'Ini string';
    c := 'A';
selesai.
```

Output

```
Parse success, code error free
<program>
|- <program-header>
|  |- KEYWORD(program)
|  |- IDENTIFIER(test2)
|  L- SEMICOLON(;)
|- <declaration-part>
|  L- <var-declaration>
|     |- KEYWORD(variabel)
|     |- <identifier-list>
|     |  L- IDENTIFIER(s)
```

```
|      |- COLON(:)
|      |- <type>
|      |  L- IDENTIFIER(string)
|      |- SEMICOLON(;)
|      |- <identifier-list>
|      |  L- IDENTIFIER(c)
|      |- COLON(:)
|      |- <type>
|      |  L- KEYWORD(char)
|      L- SEMICOLON(;)
|- <compound-statement>
|  |- KEYWORD(mulai)
|  |- <statement-list>
|  |  |- <statement>
|  |  |  L- <assignment-statement>
|  |  |     |- <variable>
|  |  |     |  L- IDENTIFIER(s)
|  |  |     |- ASSIGN_OPERATOR(:=)
|  |  |     L- <expression>
|  |  |        L- <simple-expression>
|  |  |           L- <term>
|  |  |              L- <factor>
|  |  |                 L- STRING_LITERAL('Ini string')
|  |  |- SEMICOLON(;)
|  |  |- <statement>
|  |  |  L- <assignment-statement>
|  |  |     |- <variable>
|  |  |     |  L- IDENTIFIER(c)
|  |  |     |- ASSIGN_OPERATOR(:=)
|  |  |     L- <expression>
|  |  |        L- <simple-expression>
|  |  |           L- <term>
|  |  |              L- <factor>
|  |  |                 L- CHAR_LITERAL('A')
|  |  |- SEMICOLON(;)
|  |  L- <statement>
|  L- KEYWORD(selesai)
L- DOT(.)
```

Kasus 8

Input file: test3.pas

```
program test3;

variabel
    a, b: integer;

mulai
    a := 7;
    b := 3;

    a := a + b;
    a := a - b;
    a := a * b;
```

```
    a := a / b;
    a := a bagi b;
    a := a mod b;

    jika a = b maka
        a := 0
    selainitu
        a := -1;

    jika a <> b maka
        a := 1
    selainitu
        a := -1;

    jika a <= b maka
        a := 2
    selainitu
        a := -2;

    jika a >= b maka
        a := 3
    selainitu
        a := -3;
selesai.
```

Output

```
<program>
|- <program-header>
|  |- KEYWORD(program)
|  |- IDENTIFIER(test3)
|  L- SEMICOLON(;)
|- <declaration-part>
|  L- <var-declaration>
|     |- KEYWORD(variabel)
|     |- <identifier-list>
|     |  |- IDENTIFIER(a)
|     |  |- COMMA(,)
|     |  L- IDENTIFIER(b)
|     |- COLON(:)
|     |- <type>
|     |  L- KEYWORD(integer)
|     L- SEMICOLON(;)
|- <compound-statement>
|  |- KEYWORD(mulai)
|  |- <statement-list>
|  |  |- <statement>
|  |  |  L- <assignment-statement>
|  |  |     |- <variable>
|  |  |     |  L- IDENTIFIER(a)
|  |  |     |- ASSIGN_OPERATOR(:=)
|  |  |     L- <expression>
|  |  |        L- <simple-expression>
|  |  |           L- <term>
|  |  |              L- <factor>
```

```
|   |   |                 L- NUMBER(7)
|   |   |- SEMICOLON(;)
|   |   |- <statement>
|   |   |  L- <assignment-statement>
|   |   |     |- <variable>
|   |   |     |  L- IDENTIFIER(b)
|   |   |     |- ASSIGN_OPERATOR(:=)
|   |   |     L- <expression>
|   |   |        L- <simple-expression>
|   |   |           L- <term>
|   |   |              L- <factor>
|   |   |                 L- NUMBER(3)
|   |   |- SEMICOLON(;)
|   |   |- <statement>
|   |   |  L- <assignment-statement>
|   |   |     |- <variable>
|   |   |     |  L- IDENTIFIER(a)
|   |   |     |- ASSIGN_OPERATOR(:=)
|   |   |     L- <expression>
|   |   |        L- <simple-expression>
|   |   |           |- <term>
|   |   |           |  L- <factor>
|   |   |           |     L- <variable>
|   |   |           |        L- IDENTIFIER(a)
|   |   |           |- ARITHMETIC_OPERATOR(+)
|   |   |           L- <term>
|   |   |              L- <factor>
|   |   |                 L- <variable>
|   |   |                    L- IDENTIFIER(b)
|   |   |- SEMICOLON(;)
|   |   |- <statement>
|   |   |  L- <assignment-statement>
|   |   |     |- <variable>
|   |   |     |  L- IDENTIFIER(a)
|   |   |     |- ASSIGN_OPERATOR(:=)
|   |   |     L- <expression>
|   |   |        L- <simple-expression>
|   |   |           |- <term>
|   |   |           |  L- <factor>
|   |   |           |     L- <variable>
|   |   |           |        L- IDENTIFIER(a)
|   |   |           |- ARITHMETIC_OPERATOR(-)
|   |   |           L- <term>
|   |   |              L- <factor>
|   |   |                 L- <variable>
|   |   |                    L- IDENTIFIER(b)
|   |   |- SEMICOLON(;)
|   |   |- <statement>
|   |   |  L- <assignment-statement>
|   |   |     |- <variable>
|   |   |     |  L- IDENTIFIER(a)
|   |   |     |- ASSIGN_OPERATOR(:=)
|   |   |     L- <expression>
|   |   |        L- <simple-expression>
|   |   |           L- <term>
|   |   |              |- <factor>
```

```
|  |  |              |  L- <variable>
|  |  |              |     L- IDENTIFIER(a)
|  |  |              |- <multiplicative-operator>
|  |  |              |  L- ARITHMETIC_OPERATOR(*)
|  |  |              L- <factor>
|  |  |                 L- <variable>
|  |  |                    L- IDENTIFIER(b)
|  |  |- SEMICOLON(;)
|  |  |- <statement>
|  |  |  L- <assignment-statement>
|  |  |     |- <variable>
|  |  |     |  L- IDENTIFIER(a)
|  |  |     |- ASSIGN_OPERATOR(:=)
|  |  |     L- <expression>
|  |  |        L- <simple-expression>
|  |  |           L- <term>
|  |  |              |- <factor>
|  |  |              |  L- <variable>
|  |  |              |     L- IDENTIFIER(a)
|  |  |              |- <multiplicative-operator>
|  |  |              |  L- ARITHMETIC_OPERATOR(/)
|  |  |              L- <factor>
|  |  |                 L- <variable>
|  |  |                    L- IDENTIFIER(b)
|  |  |- SEMICOLON(;)
|  |  |- <statement>
|  |  |  L- <assignment-statement>
|  |  |     |- <variable>
|  |  |     |  L- IDENTIFIER(a)
|  |  |     |- ASSIGN_OPERATOR(:=)
|  |  |     L- <expression>
|  |  |        L- <simple-expression>
|  |  |           L- <term>
|  |  |              |- <factor>
|  |  |              |  L- <variable>
|  |  |              |     L- IDENTIFIER(a)
|  |  |              |- <multiplicative-operator>
|  |  |              |  L- ARITHMETIC_OPERATOR(bagi)
|  |  |              L- <factor>
|  |  |                 L- <variable>
|  |  |                    L- IDENTIFIER(b)
|  |  |- SEMICOLON(;)
|  |  |- <statement>
|  |  |  L- <assignment-statement>
|  |  |     |- <variable>
|  |  |     |  L- IDENTIFIER(a)
|  |  |     |- ASSIGN_OPERATOR(:=)
|  |  |     L- <expression>
|  |  |        L- <simple-expression>
|  |  |           L- <term>
|  |  |              |- <factor>
|  |  |              |  L- <variable>
|  |  |              |     L- IDENTIFIER(a)
|  |  |              |- <multiplicative-operator>
|  |  |              |  L- ARITHMETIC_OPERATOR(mod)
|  |  |              L- <factor>
```

```
|  |  |                 L- <variable>
|  |  |                    L- IDENTIFIER(b)
|  |  |- SEMICOLON(;)
|  |  |- <statement>
|  |  |  L- <if-statement>
|  |  |     |- KEYWORD(jika)
|  |  |     |- <expression>
|  |  |     |  |- <simple-expression>
|  |  |     |  |  L- <term>
|  |  |     |  |     L- <factor>
|  |  |     |  |        L- <variable>
|  |  |     |  |           L- IDENTIFIER(a)
|  |  |     |  |- <relational-operator>
|  |  |     |  |  L- RELATIONAL_OPERATOR(=)
|  |  |     |  L- <simple-expression>
|  |  |     |     L- <term>
|  |  |     |        L- <factor>
|  |  |     |           L- <variable>
|  |  |     |              L- IDENTIFIER(b)
|  |  |     |- KEYWORD(maka)
|  |  |     |- <statement>
|  |  |     |  L- <assignment-statement>
|  |  |     |     |- <variable>
|  |  |     |     |  L- IDENTIFIER(a)
|  |  |     |     |- ASSIGN_OPERATOR(:=)
|  |  |     |     L- <expression>
|  |  |     |        L- <simple-expression>
|  |  |     |           L- <term>
|  |  |     |              L- <factor>
|  |  |     |                 L- NUMBER(0)
|  |  |     |- KEYWORD(selainitu)
|  |  |     L- <statement>
|  |  |        L- <assignment-statement>
|  |  |           |- <variable>
|  |  |           |  L- IDENTIFIER(a)
|  |  |           |- ASSIGN_OPERATOR(:=)
|  |  |           L- <expression>
|  |  |              L- <simple-expression>
|  |  |                 |- ARITHMETIC_OPERATOR(-)
|  |  |                 L- <term>
|  |  |                    L- <factor>
|  |  |                       L- NUMBER(1)
|  |  |- SEMICOLON(;)
|  |  |- <statement>
|  |  |  L- <if-statement>
|  |  |     |- KEYWORD(jika)
|  |  |     |- <expression>
|  |  |     |  |- <simple-expression>
|  |  |     |  |  L- <term>
|  |  |     |  |     L- <factor>
|  |  |     |  |        L- <variable>
|  |  |     |  |           L- IDENTIFIER(a)
|  |  |     |  |- <relational-operator>
|  |  |     |  |  L- RELATIONAL_OPERATOR(<>)
|  |  |     |  L- <simple-expression>
|  |  |     |     L- <term>
```

```
|  |  |     |        L- <factor>
|  |  |     |           L- <variable>
|  |  |     |              L- IDENTIFIER(b)
|  |  |     |- KEYWORD(maka)
|  |  |     |- <statement>
|  |  |     |  L- <assignment-statement>
|  |  |     |     |- <variable>
|  |  |     |     |  L- IDENTIFIER(a)
|  |  |     |     |- ASSIGN_OPERATOR(:=)
|  |  |     |     L- <expression>
|  |  |     |        L- <simple-expression>
|  |  |     |           L- <term>
|  |  |     |              L- <factor>
|  |  |     |                 L- NUMBER(1)
|  |  |     |- KEYWORD(selainitu)
|  |  |     L- <statement>
|  |  |        L- <assignment-statement>
|  |  |           |- <variable>
|  |  |           |  L- IDENTIFIER(a)
|  |  |           |- ASSIGN_OPERATOR(:=)
|  |  |           L- <expression>
|  |  |              L- <simple-expression>
|  |  |                 |- ARITHMETIC_OPERATOR(-)
|  |  |                 L- <term>
|  |  |                    L- <factor>
|  |  |                       L- NUMBER(1)
|  |  |- SEMICOLON(;)
|  |  |- <statement>
|  |  |  L- <if-statement>
|  |  |     |- KEYWORD(jika)
|  |  |     |- <expression>
|  |  |     |  |- <simple-expression>
|  |  |     |  |  L- <term>
|  |  |     |  |     L- <factor>
|  |  |     |  |        L- <variable>
|  |  |     |  |           L- IDENTIFIER(a)
|  |  |     |  |- <relational-operator>
|  |  |     |  |  L- RELATIONAL_OPERATOR(<=)
|  |  |     |  L- <simple-expression>
|  |  |     |     L- <term>
|  |  |     |        L- <factor>
|  |  |     |           L- <variable>
|  |  |     |              L- IDENTIFIER(b)
|  |  |     |- KEYWORD(maka)
|  |  |     |- <statement>
|  |  |     |  L- <assignment-statement>
|  |  |     |     |- <variable>
|  |  |     |     |  L- IDENTIFIER(a)
|  |  |     |     |- ASSIGN_OPERATOR(:=)
|  |  |     |     L- <expression>
|  |  |     |        L- <simple-expression>
|  |  |     |           L- <term>
|  |  |     |              L- <factor>
|  |  |     |                 L- NUMBER(2)
|  |  |     |- KEYWORD(selainitu)
|  |  |     L- <statement>
```

```
|  |  |        L- <assignment-statement>
|  |  |           |- <variable>
|  |  |           |  L- IDENTIFIER(a)
|  |  |           |- ASSIGN_OPERATOR(:=)
|  |  |           L- <expression>
|  |  |              L- <simple-expression>
|  |  |                 |- ARITHMETIC_OPERATOR(-)
|  |  |                 L- <term>
|  |  |                    L- <factor>
|  |  |                       L- NUMBER(2)
|  |  |- SEMICOLON(;)
|  |  |- <statement>
|  |  |  L- <if-statement>
|  |  |     |- KEYWORD(jika)
|  |  |     |- <expression>
|  |  |     |  |- <simple-expression>
|  |  |     |  |  L- <term>
|  |  |     |  |     L- <factor>
|  |  |     |  |        L- <variable>
|  |  |     |  |           L- IDENTIFIER(a)
|  |  |     |  |- <relational-operator>
|  |  |     |  |  L- RELATIONAL_OPERATOR(>=)
|  |  |     |  L- <simple-expression>
|  |  |     |     L- <term>
|  |  |     |        L- <factor>
|  |  |     |           L- <variable>
|  |  |     |              L- IDENTIFIER(b)
|  |  |     |- KEYWORD(maka)
|  |  |     |- <statement>
|  |  |     |  L- <assignment-statement>
|  |  |     |     |- <variable>
|  |  |     |     |  L- IDENTIFIER(a)
|  |  |     |     |- ASSIGN_OPERATOR(:=)
|  |  |     |     L- <expression>
|  |  |     |        L- <simple-expression>
|  |  |     |           L- <term>
|  |  |     |              L- <factor>
|  |  |     |                 L- NUMBER(3)
|  |  |     |- KEYWORD(selainitu)
|  |  |     L- <statement>
|  |  |        L- <assignment-statement>
|  |  |           |- <variable>
|  |  |           |  L- IDENTIFIER(a)
|  |  |           |- ASSIGN_OPERATOR(:=)
|  |  |           L- <expression>
|  |  |              L- <simple-expression>
|  |  |                 |- ARITHMETIC_OPERATOR(-)
|  |  |                 L- <term>
|  |  |                    L- <factor>
|  |  |                       L- NUMBER(3)
|  |  |- SEMICOLON(;)
|  |  L- <statement>
|  L- KEYWORD(selesai)
L- DOT(.)
```

Kasus 9

Input file: test4.pas

```
program test4;

variabel
    x, y: integer;
    hasil: boolean;

mulai
    x := 10;
    y := 5;


    jika x < y maka
        x := 1
    selainitu
        x := 0;

    selama x < 5 lakukan
        x := x + 1;

    untuk x := 1 ke 3 lakukan
        y := y + x;
selesai.
```

Output

```
<program>
|- <program-header>
|  |- KEYWORD(program)
|  |- IDENTIFIER(test4)
|  L- SEMICOLON(;)
|- <declaration-part>
|  L- <var-declaration>
|     |- KEYWORD(variabel)
|     |- <identifier-list>
|     |  |- IDENTIFIER(x)
|     |  |- COMMA(,)
|     |  L- IDENTIFIER(y)
|     |- COLON(:)
|     |- <type>
|     |  L- KEYWORD(integer)
|     |- SEMICOLON(;)
|     |- <identifier-list>
|     |  L- IDENTIFIER(hasil)
|     |- COLON(:)
|     |- <type>
|     |  L- KEYWORD(boolean)
|     L- SEMICOLON(;)
|- <compound-statement>
|  |- KEYWORD(mulai)
|  |- <statement-list>
|  |  |- <statement>
|  |  |  L- <assignment-statement>
```

```
|   |   |      |- <variable>
|   |   |      |  L- IDENTIFIER(x)
|   |   |      |- ASSIGN_OPERATOR(:=)
|   |   |      L- <expression>
|   |   |         L- <simple-expression>
|   |   |            L- <term>
|   |   |               L- <factor>
|   |   |                  L- NUMBER(10)
|   |   |- SEMICOLON(;)
|   |   |- <statement>
|   |   |  L- <assignment-statement>
|   |   |      |- <variable>
|   |   |      |  L- IDENTIFIER(y)
|   |   |      |- ASSIGN_OPERATOR(:=)
|   |   |      L- <expression>
|   |   |         L- <simple-expression>
|   |   |            L- <term>
|   |   |               L- <factor>
|   |   |                  L- NUMBER(5)
|   |   |- SEMICOLON(;)
|   |   |- <statement>
|   |   |  L- <if-statement>
|   |   |      |- KEYWORD(jika)
|   |   |      |- <expression>
|   |   |      |  |- <simple-expression>
|   |   |      |  |  L- <term>
|   |   |      |  |     L- <factor>
|   |   |      |  |        L- <variable>
|   |   |      |  |           L- IDENTIFIER(x)
|   |   |      |  |- <relational-operator>
|   |   |      |  |  L- RELATIONAL_OPERATOR(<)
|   |   |      |  L- <simple-expression>
|   |   |      |     L- <term>
|   |   |      |        L- <factor>
|   |   |      |           L- <variable>
|   |   |      |              L- IDENTIFIER(y)
|   |   |      |- KEYWORD(maka)
|   |   |      |- <statement>
|   |   |      |  L- <assignment-statement>
|   |   |      |     |- <variable>
|   |   |      |     |  L- IDENTIFIER(x)
|   |   |      |     |- ASSIGN_OPERATOR(:=)
|   |   |      |     L- <expression>
|   |   |      |        L- <simple-expression>
|   |   |      |           L- <term>
|   |   |      |              L- <factor>
|   |   |      |                 L- NUMBER(1)
|   |   |      |- KEYWORD(selainitu)
|   |   |      L- <statement>
|   |   |         L- <assignment-statement>
|   |   |            |- <variable>
|   |   |            |  L- IDENTIFIER(x)
|   |   |            |- ASSIGN_OPERATOR(:=)
|   |   |            L- <expression>
|   |   |               L- <simple-expression>
|   |   |                  L- <term>
```

```
|   |   |                     L- <factor>
|   |   |                        L- NUMBER(0)
|   |   |- SEMICOLON(;)
|   |   |- <statement>
|   |   |  L- <while-statement>
|   |   |     |- KEYWORD(selama)
|   |   |     |- <expression>
|   |   |     |  |- <simple-expression>
|   |   |     |  |  L- <term>
|   |   |     |  |     L- <factor>
|   |   |     |  |        L- <variable>
|   |   |     |  |           L- IDENTIFIER(x)
|   |   |     |  |- <relational-operator>
|   |   |     |  |  L- RELATIONAL_OPERATOR(<)
|   |   |     |  L- <simple-expression>
|   |   |     |     L- <term>
|   |   |     |        L- <factor>
|   |   |     |           L- NUMBER(5)
|   |   |     |- KEYWORD(lakukan)
|   |   |     L- <statement>
|   |   |        L- <assignment-statement>
|   |   |           |- <variable>
|   |   |           |  L- IDENTIFIER(x)
|   |   |           |- ASSIGN_OPERATOR(:=)
|   |   |           L- <expression>
|   |   |              L- <simple-expression>
|   |   |                 |- <term>
|   |   |                 |  L- <factor>
|   |   |                 |     L- <variable>
|   |   |                 |        L- IDENTIFIER(x)
|   |   |                 |- ARITHMETIC_OPERATOR(+)
|   |   |                 L- <term>
|   |   |                    L- <factor>
|   |   |                       L- NUMBER(1)
|   |   |- SEMICOLON(;)
|   |   |- <statement>
|   |   |  L- <for-statement>
|   |   |     |- KEYWORD(untuk)
|   |   |     |- IDENTIFIER(x)
|   |   |     |- ASSIGN_OPERATOR(:=)
|   |   |     |- <expression>
|   |   |     |  L- <simple-expression>
|   |   |     |     L- <term>
|   |   |     |        L- <factor>
|   |   |     |           L- NUMBER(1)
|   |   |     |- KEYWORD(ke)
|   |   |     |- <expression>
|   |   |     |  L- <simple-expression>
|   |   |     |     L- <term>
|   |   |     |        L- <factor>
|   |   |     |           L- NUMBER(3)
|   |   |     |- KEYWORD(lakukan)
|   |   |     L- <statement>
|   |   |        L- <assignment-statement>
|   |   |           |- <variable>
|   |   |           |  L- IDENTIFIER(y)
```

```
|  |  |           |- ASSIGN_OPERATOR(:=)
|  |  |           L- <expression>
|  |  |              L- <simple-expression>
|  |  |                 |- <term>
|  |  |                 |  L- <factor>
|  |  |                 |     L- <variable>
|  |  |                 |        L- IDENTIFIER(y)
|  |  |                 |- ARITHMETIC_OPERATOR(+)
|  |  |                 L- <term>
|  |  |                    L- <factor>
|  |  |                       L- <variable>
|  |  |                          L- IDENTIFIER(x)
|  |  |- SEMICOLON(;)
|  |  L- <statement>
|  L- KEYWORD(selesai)
L- DOT(.)
```

Kasus 10

Input file: test5_error.pas

```
program UjiKasus;

variabel
  x : integer;

mulai
  x := 2;

  kasus x dari
    1: satu;
    2: dua;
    3: tiga;
  akhir;

selesai.
```

Output

```
Error, expected ASSIGN_OPERATOR(:=) but got SEMICOLON(;)
Exiting program
```

Kasus 11

Input file: test6.pas

```
program TesAssign;

variabel
  x : integer;
  satu : integer;

mulai
  x := 1;
```

```
  kasus x dari
    1: satu := 1;
    2: satu := 100;
  akhir;

selesai.
```

Output

```
<program>
|- <program-header>
|  |- KEYWORD(program)
|  |- IDENTIFIER(TesAssign)
|  L- SEMICOLON(;)
|- <declaration-part>
|  L- <var-declaration>
|     |- KEYWORD(variabel)
|     |- <identifier-list>
|     |  L- IDENTIFIER(x)
|     |- COLON(:)
|     |- <type>
|     |  L- KEYWORD(integer)
|     |- SEMICOLON(;)
|     |- <identifier-list>
|     |  L- IDENTIFIER(satu)
|     |- COLON(:)
|     |- <type>
|     |  L- KEYWORD(integer)
|     L- SEMICOLON(;)
|- <compound-statement>
|  |- KEYWORD(mulai)
|  |- <statement-list>
|  |  |- <statement>
|  |  |  L- <assignment-statement>
|  |  |     |- <variable>
|  |  |     |  L- IDENTIFIER(x)
|  |  |     |- ASSIGN_OPERATOR(:=)
|  |  |     L- <expression>
|  |  |        L- <simple-expression>
|  |  |           L- <term>
|  |  |              L- <factor>
|  |  |                 L- NUMBER(1)
|  |  |- SEMICOLON(;)
|  |  |- <statement>
|  |  |  L- <case_statement>
|  |  |     |- KEYWORD(kasus)
|  |  |     |- <expression>
|  |  |     |  L- <simple-expression>
|  |  |     |     L- <term>
|  |  |     |        L- <factor>
|  |  |     |           L- <variable>
|  |  |     |              L- IDENTIFIER(x)
|  |  |     |- KEYWORD(dari)
|  |  |     |- <const>
|  |  |     |  L- NUMBER(1)
```

```
|  |  |     |- COLON(:)
|  |  |     |- <statement>
|  |  |     |  L- <assignment-statement>
|  |  |     |     |- <variable>
|  |  |     |     |  L- IDENTIFIER(satu)
|  |  |     |     |- ASSIGN_OPERATOR(:=)
|  |  |     |     L- <expression>
|  |  |     |        L- <simple-expression>
|  |  |     |           L- <term>
|  |  |     |              L- <factor>
|  |  |     |                 L- NUMBER(1)
|  |  |     |- SEMICOLON(;)
|  |  |     |- <const>
|  |  |     |  L- NUMBER(2)
|  |  |     |- COLON(:)
|  |  |     |- <statement>
|  |  |     |  L- <assignment-statement>
|  |  |     |     |- <variable>
|  |  |     |     |  L- IDENTIFIER(satu)
|  |  |     |     |- ASSIGN_OPERATOR(:=)
|  |  |     |     L- <expression>
|  |  |     |        L- <simple-expression>
|  |  |     |           L- <term>
|  |  |     |              L- <factor>
|  |  |     |                 L- NUMBER(100)
|  |  |     |- SEMICOLON(;)
|  |  |     L- KEYWORD(akhir)
|  |  |- SEMICOLON(;)
|  |  L- <statement>
|  L- KEYWORD(selesai)
L- DOT(.)
```

Kasus 12

Input file: test7_error.pas

```
program TesKasus5;

tipe
  Data = rekaman
    kode : integer;
    nilai : integer;
  selesai;

variabel
  p : integer;
  d : char;

mulai
  p := 3;
  d.kode := 0;
  d.nilai := 0;

  kasus p dari
    1: satu;
    2: dua;
```

```
    3, 4, 5: blok1;
    10: blok2;
    -1: minusSatu;
  akhir;

selesai.
```

Output

```
Error, expected ASSIGN_OPERATOR(:=) but got SEMICOLON(;)
Exiting program
```

Kasus 13

Input file: test8.pas

```
program Tes111213142122;

variabel
  radius: integer;
  area, res1, res2, res3: real;

prosedur namaProsedur1(r: integer; a: real);
  konstanta
    pi := 3;

  mulai
    jika r < 5 maka
      a := 75
    selainitu
      a := pi * r * r;
  selesai;

prosedur namaProsedur2;
  mulai
    writeln('Testing prosedur without paramater');
  selesai;

fungsi namaFungsi1(r: integer) : real;
  konstanta
    pi := 3;

  mulai
    jika r < 5 maka
      namaFungsi1 := 75
    selainitu
      namaFungsi1 := pi * r * r;
  selesai;

fungsi namaFungsi2 : real;
  mulai
    namaFungsi2 := 75
  selesai;

mulai
```

```
  radius := 5;

  namaProsedur1(radius, res1);

namaProsedur2(harusnyaBisaTanpaParameterTapiKarenaDiSpekSemuaProecedureDanFunc
tionCallHarusPakeParenthesisJadiKalimatIniDigunakanSebagaiFillerSupayaValid);

  res2 := namaFungsi1(radius);
  res3 :=
namaFungsi2(harusnyaBisaTanpaParameterTapiKarenaDiSpekSemuaProecedureDanFuncti
onCallHarusPakeParenthesisJadiKalimatIniDigunakanSebagaiFillerSupayaValid);
selesai.
```

Output

```
<program>
|- <program-header>
|  |- KEYWORD(program)
|  |- IDENTIFIER(Tes111213142122)
|  L- SEMICOLON(;)
|- <declaration-part>
|  |- <var-declaration>
|  |  |- KEYWORD(variabel)
|  |  |- <identifier-list>
|  |  |  L- IDENTIFIER(radius)
|  |  |- COLON(:)
|  |  |- <type>
|  |  |  L- KEYWORD(integer)
|  |  |- SEMICOLON(;)
|  |  |- <identifier-list>
|  |  |  |- IDENTIFIER(area)
|  |  |  |- COMMA(,)
|  |  |  |- IDENTIFIER(res1)
|  |  |  |- COMMA(,)
|  |  |  |- IDENTIFIER(res2)
|  |  |  |- COMMA(,)
|  |  |  L- IDENTIFIER(res3)
|  |  |- COLON(:)
|  |  |- <type>
|  |  |  L- KEYWORD(real)
|  |  L- SEMICOLON(;)
|  |- <subprogram-declaration>
|  |  L- <procedure-declaration>
|  |     |- KEYWORD(prosedur)
|  |     |- IDENTIFIER(namaProsedur1)
|  |     |- <formal-parameter-list>
|  |     |  |- LPARENTHESIS(()
|  |     |  |- <parameter-group>
|  |     |  |  |- <identifier-list>
|  |     |  |  |  L- IDENTIFIER(r)
|  |     |  |  |- COLON(:)
|  |     |  |  L- <type>
|  |     |  |     L- KEYWORD(integer)
|  |     |  |- SEMICOLON(;)
|  |     |  |- <parameter-group>
```

```
|  |     |  |  |- <identifier-list>
|  |     |  |  |  L- IDENTIFIER(a)
|  |     |  |  |- COLON(:)
|  |     |  |  L- <type>
|  |     |  |     L- KEYWORD(real)
|  |     |  L- RPARENTHESIS())
|  |     |- SEMICOLON(;)
|  |     |- <declaration-part>
|  |     |  L- <const-declaration>
|  |     |     |- KEYWORD(konstanta)
|  |     |     |- IDENTIFIER(pi)
|  |     |     |- RELATIONAL_OPERATOR(=)
|  |     |     |- <const>
|  |     |     |  L- <number-statement>
|  |     |     |     L- NUMBER(3)
|  |     |     L- SEMICOLON(;)
|  |     |- <compound-statement>
|  |     |  |- KEYWORD(mulai)
|  |     |  |- <statement-list>
|  |     |  |  |- <statement>
|  |     |  |  |  L- <if-statement>
|  |     |  |  |     |- KEYWORD(jika)
|  |     |  |  |     |- <expression>
|  |     |  |  |     |  |- <simple-expression>
|  |     |  |  |     |  |  L- <term>
|  |     |  |  |     |  |     L- <factor>
|  |     |  |  |     |  |        L- <variable>
|  |     |  |  |     |  |           L- IDENTIFIER(r)
|  |     |  |  |     |  |- <relational-operator>
|  |     |  |  |     |  |  L- RELATIONAL_OPERATOR(<)
|  |     |  |  |     |  L- <simple-expression>
|  |     |  |  |     |     L- <term>
|  |     |  |  |     |        L- <factor>
|  |     |  |  |     |           L- <number-statement>
|  |     |  |  |     |              L- NUMBER(5)
|  |     |  |  |     |- KEYWORD(maka)
|  |     |  |  |     |- <statement>
|  |     |  |  |     |  L- <assignment-statement>
|  |     |  |  |     |     |- <variable>
|  |     |  |  |     |     |  L- IDENTIFIER(a)
|  |     |  |  |     |     |- ASSIGN_OPERATOR(:=)
|  |     |  |  |     |     L- <expression>
|  |     |  |  |     |        L- <simple-expression>
|  |     |  |  |     |           L- <term>
|  |     |  |  |     |              L- <factor>
|  |     |  |  |     |                 L- <number-statement>
|  |     |  |  |     |                    L- NUMBER(75)
|  |     |  |  |     |- KEYWORD(selainitu)
|  |     |  |  |     L- <statement>
|  |     |  |  |        L- <assignment-statement>
|  |     |  |  |           |- <variable>
|  |     |  |  |           |  L- IDENTIFIER(a)
|  |     |  |  |           |- ASSIGN_OPERATOR(:=)
|  |     |  |  |           L- <expression>
|  |     |  |  |              L- <simple-expression>
|  |     |  |  |                 L- <term>
```

```
|  |     |  |  |                    |- <factor>
|  |     |  |  |                    |  L- <variable>
|  |     |  |  |                    |     L- IDENTIFIER(pi)
|  |     |  |  |                    |- <multiplicative-operator>
|  |     |  |  |                    |  L- ARITHMETIC_OPERATOR(*)
|  |     |  |  |                    |- <factor>
|  |     |  |  |                    |  L- <variable>
|  |     |  |  |                    |     L- IDENTIFIER(r)
|  |     |  |  |                    |- <multiplicative-operator>
|  |     |  |  |                    |  L- ARITHMETIC_OPERATOR(*)
|  |     |  |  |                    L- <factor>
|  |     |  |  |                       L- <variable>
|  |     |  |  |                          L- IDENTIFIER(r)
|  |     |  |  |- SEMICOLON(;)
|  |     |  |  L- <statement>
|  |     |  L- KEYWORD(selesai)
|  |     L- SEMICOLON(;)
|  |- <subprogram-declaration>
|  |  L- <procedure-declaration>
|  |     |- KEYWORD(prosedur)
|  |     |- IDENTIFIER(namaProsedur2)
|  |     |- SEMICOLON(;)
|  |     |- <declaration-part>
|  |     |- <compound-statement>
|  |     |  |- KEYWORD(mulai)
|  |     |  |- <statement-list>
|  |     |  |  |- <statement>
|  |     |  |  |  L- <procedure/function-call>
|  |     |  |  |     |- IDENTIFIER(writeln)
|  |     |  |  |     |- LPARENTHESIS(()
|  |     |  |  |     |- <parameter-list>
|  |     |  |  |     |  L- <expression>
|  |     |  |  |     |     L- <simple-expression>
|  |     |  |  |     |        L- <term>
|  |     |  |  |     |           L- <factor>
|  |     |  |  |     |              L- STRING_LITERAL('Testing prosedur
without paramater')
|  |     |  |  |     L- RPARENTHESIS())
|  |     |  |  |- SEMICOLON(;)
|  |     |  |  L- <statement>
|  |     |  L- KEYWORD(selesai)
|  |     L- SEMICOLON(;)
|  |- <subprogram-declaration>
|  |  L- <function-declaration>
|  |     |- KEYWORD(fungsi)
|  |     |- IDENTIFIER(namaFungsi1)
|  |     |- <formal-parameter-list>
|  |     |  |- LPARENTHESIS(()
|  |     |  |- <parameter-group>
|  |     |  |  |- <identifier-list>
|  |     |  |  |  L- IDENTIFIER(r)
|  |     |  |  |- COLON(:)
|  |     |  |  L- <type>
|  |     |  |     L- KEYWORD(integer)
|  |     |  L- RPARENTHESIS())
|  |     |- COLON(:)
```

```
|   |      |- <type>
|   |      |  L- KEYWORD(real)
|   |      |- SEMICOLON(;)
|   |      |- <declaration-part>
|   |      |  L- <const-declaration>
|   |      |     |- KEYWORD(konstanta)
|   |      |     |- IDENTIFIER(pi)
|   |      |     |- RELATIONAL_OPERATOR(=)
|   |      |     |- <const>
|   |      |     |  L- <number-statement>
|   |      |     |     L- NUMBER(3)
|   |      |     L- SEMICOLON(;)
|   |      |- <compound-statement>
|   |      |  |- KEYWORD(mulai)
|   |      |  |- <statement-list>
|   |      |  |  |- <statement>
|   |      |  |  |  L- <if-statement>
|   |      |  |  |     |- KEYWORD(jika)
|   |      |  |  |     |- <expression>
|   |      |  |  |     |  |- <simple-expression>
|   |      |  |  |     |  |  L- <term>
|   |      |  |  |     |  |     L- <factor>
|   |      |  |  |     |  |        L- <variable>
|   |      |  |  |     |  |           L- IDENTIFIER(r)
|   |      |  |  |     |  |- <relational-operator>
|   |      |  |  |     |  |  L- RELATIONAL_OPERATOR(<)
|   |      |  |  |     |  L- <simple-expression>
|   |      |  |  |     |     L- <term>
|   |      |  |  |     |        L- <factor>
|   |      |  |  |     |           L- <number-statement>
|   |      |  |  |     |              L- NUMBER(5)
|   |      |  |  |     |- KEYWORD(maka)
|   |      |  |  |     |- <statement>
|   |      |  |  |     |  L- <assignment-statement>
|   |      |  |  |     |     |- <variable>
|   |      |  |  |     |     |  L- IDENTIFIER(namaFungsi1)
|   |      |  |  |     |     |- ASSIGN_OPERATOR(:=)
|   |      |  |  |     |     L- <expression>
|   |      |  |  |     |        L- <simple-expression>
|   |      |  |  |     |           L- <term>
|   |      |  |  |     |              L- <factor>
|   |      |  |  |     |                 L- <number-statement>
|   |      |  |  |     |                    L- NUMBER(75)
|   |      |  |  |     |- KEYWORD(selainitu)
|   |      |  |  |     L- <statement>
|   |      |  |  |        L- <assignment-statement>
|   |      |  |  |           |- <variable>
|   |      |  |  |           |  L- IDENTIFIER(namaFungsi1)
|   |      |  |  |           |- ASSIGN_OPERATOR(:=)
|   |      |  |  |           L- <expression>
|   |      |  |  |              L- <simple-expression>
|   |      |  |  |                 L- <term>
|   |      |  |  |                    |- <factor>
|   |      |  |  |                    |  L- <variable>
|   |      |  |  |                    |     L- IDENTIFIER(pi)
|   |      |  |  |                    |- <multiplicative-operator>
```

```
|  |     |  |  |                    |  L- ARITHMETIC_OPERATOR(*)
|  |     |  |  |                    |- <factor>
|  |     |  |  |                    |  L- <variable>
|  |     |  |  |                    |     L- IDENTIFIER(r)
|  |     |  |  |                    |- <multiplicative-operator>
|  |     |  |  |                    |  L- ARITHMETIC_OPERATOR(*)
|  |     |  |  |                    L- <factor>
|  |     |  |  |                       L- <variable>
|  |     |  |  |                          L- IDENTIFIER(r)
|  |     |  |  |- SEMICOLON(;)
|  |     |  |  L- <statement>
|  |     |  L- KEYWORD(selesai)
|  |     L- SEMICOLON(;)
|  L- <subprogram-declaration>
|     L- <function-declaration>
|        |- KEYWORD(fungsi)
|        |- IDENTIFIER(namaFungsi2)
|        |- COLON(:)
|        |- <type>
|        |  L- KEYWORD(real)
|        |- SEMICOLON(;)
|        |- <declaration-part>
|        |- <compound-statement>
|        |  |- KEYWORD(mulai)
|        |  |- <statement-list>
|        |  |  L- <statement>
|        |  |     L- <assignment-statement>
|        |  |        |- <variable>
|        |  |        |  L- IDENTIFIER(namaFungsi2)
|        |  |        |- ASSIGN_OPERATOR(:=)
|        |  |        L- <expression>
|        |  |           L- <simple-expression>
|        |  |              L- <term>
|        |  |                 L- <factor>
|        |  |                    L- <number-statement>
|        |  |                       L- NUMBER(75)
|        |  L- KEYWORD(selesai)
|        L- SEMICOLON(;)
|- <compound-statement>
|  |- KEYWORD(mulai)
|  |- <statement-list>
|  |  |- <statement>
|  |  |  L- <assignment-statement>
|  |  |     |- <variable>
|  |  |     |  L- IDENTIFIER(radius)
|  |  |     |- ASSIGN_OPERATOR(:=)
|  |  |     L- <expression>
|  |  |        L- <simple-expression>
|  |  |           L- <term>
|  |  |              L- <factor>
|  |  |                 L- <number-statement>
|  |  |                    L- NUMBER(5)
|  |  |- SEMICOLON(;)
|  |  |- <statement>
|  |  |  L- <procedure/function-call>
|  |  |     |- IDENTIFIER(namaProsedur1)
```

```
|   |   |      |- LPARENTHESIS(()
|   |   |      |- <parameter-list>
|   |   |      |   |- <expression>
|   |   |      |   |   L- <simple-expression>
|   |   |      |   |      L- <term>
|   |   |      |   |         L- <factor>
|   |   |      |   |            L- <variable>
|   |   |      |   |               L- IDENTIFIER(radius)
|   |   |      |   |- COMMA(,)
|   |   |      |   L- <expression>
|   |   |      |      L- <simple-expression>
|   |   |      |         L- <term>
|   |   |      |            L- <factor>
|   |   |      |               L- <variable>
|   |   |      |                  L- IDENTIFIER(res1)
|   |   |      L- RPARENTHESIS())
|   |   |- SEMICOLON(;)
|   |   |- <statement>
|   |   |   L- <procedure/function-call>
|   |   |      |- IDENTIFIER(namaProsedur2)
|   |   |      |- LPARENTHESIS(()
|   |   |      |- <parameter-list>
|   |   |      |   L- <expression>
|   |   |      |      L- <simple-expression>
|   |   |      |         L- <term>
|   |   |      |            L- <factor>
|   |   |      |               L- <variable>
|   |   |      |                  L-
IDENTIFIER(harusnyaBisaTanpaParameterTapiKarenaDiSpekSemuaProecedureDanFunctio
nCallHarusPakeParenthesisJadiKalimatIniDigunakanSebagaiFillerSupayaValid)
|   |   |      L- RPARENTHESIS())
|   |   |- SEMICOLON(;)
|   |   |- <statement>
|   |   |   L- <assignment-statement>
|   |   |      |- <variable>
|   |   |      |   L- IDENTIFIER(res2)
|   |   |      |- ASSIGN_OPERATOR(:=)
|   |   |      L- <expression>
|   |   |         L- <simple-expression>
|   |   |            L- <term>
|   |   |               L- <factor>
|   |   |                  L- <procedure/function-call>
|   |   |                     |- IDENTIFIER(namaFungsi1)
|   |   |                     |- LPARENTHESIS(()
|   |   |                     |- <parameter-list>
|   |   |                     |   L- <expression>
|   |   |                     |      L- <simple-expression>
|   |   |                     |         L- <term>
|   |   |                     |            L- <factor>
|   |   |                     |               L- <variable>
|   |   |                     |                  L- IDENTIFIER(radius)
|   |   |                     L- RPARENTHESIS())
|   |   |- SEMICOLON(;)
|   |   |- <statement>
|   |   |   L- <assignment-statement>
|   |   |      |- <variable>
```

```
|  |  |     |  L- IDENTIFIER(res3)
|  |  |     |- ASSIGN_OPERATOR(:=)
|  |  |     L- <expression>
|  |  |        L- <simple-expression>
|  |  |           L- <term>
|  |  |              L- <factor>
|  |  |                 L- <procedure/function-call>
|  |  |                    |- IDENTIFIER(namaFungsi2)
|  |  |                    |- LPARENTHESIS(()
|  |  |                    |- <parameter-list>
|  |  |                    |  L- <expression>
|  |  |                    |     L- <simple-expression>
|  |  |                    |        L- <term>
|  |  |                    |           L- <factor>
|  |  |                    |              L- <variable>
|  |  |                    |                 L-
IDENTIFIER(harusnyaBisaTanpaParameterTapiKarenaDiSpekSemuaProecedureDanFunctio
nCallHarusPakeParenthesisJadiKalimatIniDigunakanSebagaiFillerSupayaValid)
|  |  |                    L- RPARENTHESIS())
|  |  |- SEMICOLON(;)
|  |  L- <statement>
|  L- KEYWORD(selesai)
L- DOT(.)
```

KESIMPULAN DAN SARAN

Pada Milestone 2 ini, parser PASCAL-S telah berhasil diimplementasikan menggunakan metode recursive descent melalui kelas *TreeNode* dan *Parser*, yang secara efektif memetakan aturan grammar menjadi fungsi-fungsi terpisah. Berdasarkan 13 kasus pengujian, parser terbukti mampu memvalidasi beragam sintaks PASCAL-S—termasuk deklarasi *konstanta*, *tipe*, *variabel*, struktur *rekaman* dan *larik*, berbagai *statements* (seperti *if, for, case*) , serta deklarasi dan pemanggilan *subprogram* sekaligus melaporkan *syntax error*.

Saran yang bisa dibuat untuk Milestone 2 ini adalah memulai pengerjaan milestone dengan lebih awal dan mencoba untuk lebih memahami dan mengklarifikasi terkait spesifikasi pengerjaan relatif terhadap materi yang diajarkan.

LAMPIRAN

- https://github.com/jhotlann/LPH-Tubes-IF2224
- Grammar yang digunakan

  Representasi Grammar yang digunakan dalam notasi BNF:

```
<program> ::= <program-header> <declaration-part>

              <compound-statement> DOT(.)
```

```
<program_header> ::= KEYWORD(program) IDENTIFIER SEMICOLON(; )
```

```
<declaration_part> ::= {const-declaration} {type-declaration}
                       {var-declaration} {subprogram-declaration}
```

```
<const_declaration> ::= KEYWORD(konstanta)
                        (IDENTIFIER = <const> SEMICOLON)
                        {IDENTIFIER = <const> SEMICOLON}
```

```
<type-declaration> ::= KEYWORD(tipe)
                        IDENTIFIER = <type> SEMICOLON(;)
                        {IDENTIFIER = <type> SEMICOLON(;)}
```

```
<record_type> ::= KEYWORD(rekaman) <identifier-list> COLON(:)
                  <type> SEMICOLON(;)
                  {<identifier-list> COLON(:) <type> SEMICOLON(;)}
```

```
<var_declaration> ::= KEYWORD(variabel)
                      (<identifier-list> COLON(:) type SEMICOLON(;))
                      {<identifier-list> COLON(:) type SEMICOLON(;)}
```

```
<identifier_list> ::= IDENTIFIER {COMMA(,) IDENTIFIER}
```

```
<type> ::= KEYWORD(integer)
          | KEYWORD(real)
          | KEYWORD(boolean)
          | KEYWORD(char)
          | <array-type>
```

```
            | <record_type>

            | IDENTIFIER
```

```
<array-type> ::= KEYWORD(larik)

                 LBRACKET('[') <range> {COMMA(,) <range>}

                 RBRACKET(']') KEYWORD(dari) <type>
```

```
<range> ::= <expression> RANGE_OPERATOR(..) <expression>
```

```
<subprogram_declaration> ::= <procedure-declaration>

                             | <function-declaration>
```

```
<procedure-declaration> ::= KEYWORD(prosedur) IDENTIFIER

                            <optional-formal-parameter-list>
SEMICOLON

                            <block> SEMICOLON
```

```
<optional-formal-parameter-list> ::= <formal-parameter-list> | ε
```

```
<function_declaration> ::= KEYWORD(fungsi) IDENTIFIER
                           <optional-formal-parameter-list> COLON
                           type SEMICOLON block SEMICOLON
```

```
<formal_parameter_list> ::= LPARENTHESES <parameter_group>

                            {SEMICOLON <parameter_group>}

                             RPARENTHESIS
```

```
<parameter_group> ::= <identifier_list> COLON type
```

```
<compound_statement> ::= KEYWORD(mulai) <statement-list>

                         KEYWORD(selesai)
```

```
<statement_list> ::= <statement> {SEMICOLON <statement>}
```

```
<assignment_statement> ::= <variable> ASSIGN_OPERATOR

                           <expression>
```

```
<if_statement> ::= KEYWORD(jika) <expression> KEYWORD(maka)

                   <statement> <optional_else>
```

```
<optional_else> ::= KEYWORD(selainitu) <statement> | ε
```

```
<while_statement> ::= KEYWORD(selama) expression KEYWORD(lakukan)
                      statement
```

```
<for_statement> ::= KEYWORD(untuk) IDENTIFIER ASSIGN_OPERATOR(:=)

                    <expression> (KEYWORD(ke)

                    | KEYWORD(turun-ke)) <expression>

                    KEYWORD(lakukan) <statement>
```

```
<procedure_function_call> ::= IDENTIFIER ( LPARENTHESIS(()

                              parameter-list RPARENTHESIS()) )
```

```
<parameter-list> ::= <expression> {COMMA(",") expression}
```

```
<expression> ::= <simple_expression>

                | <simple_expression> <relational-operator>
                  <simple-expression>
```

```
<optional_plus_min_operator> ::=
                                ARITHMETIC_OPERATOR(+)|ARITHMETIC_OP
                                ERATOR(-)| ε
```

```
<simple_expression> ::= <optional_plus_min_operator> <term>
                        {<additive-operator> <term>}
```

```
<term> ::= <factor> {<multiplicative-operator> <factor>}
```

```
<factor> ::= <variable>
             | <procedure_function_call>
             | NUMBER
             | CHAR_LITERAL
             | STRING_LITERAL
             | LPARENTHESIS expression RPARENTHESIS
             | LOGICAL_OPERATOR("tidak") factor
```

```
<variable> ::= IDENTIFIER
             | IDENTIFIER {LBRACKET parameter-list RBRACKET}
             | IDENTIFIER {DOT IDENTIFIER}
```

```
<relational_operator> ::= "=" | "<>" | "<" | "<=" | ">" | ">="
```

```
<additive_operator> ::= "+" | "-" | KEYWORD(atau)
```

```
<multiplicative_operator> ::= "*" | "/"
                              | KEYWORD(bagi)
                              | KEYWORD(mod)
                              | KEYWORD(dan)
```

```
<statement> ::= <procedure_function_call>
                | <assignment-statement>
                | <compound-statement>
                | <if-statement>
                | <case-statement>
```

```
                  | <while-statement>
                  | <repeat-statement>
                  | <for-statement>
                  | ϵ
```

```
<case-statement> ::= KEYWORD(kasus) <expression> KEYWORD(dari)
       (const {COMMA(,) const} COLON(:) <statement>)
       {SEMICOLON(;) const {COMMA(,) const} COLON(:) <statement>}
       KEYWORD(akhir)
```

```
<const> ::= <signed-number>
            | NUMBER
            | IDENTIFIER
            | CHAR_LITERAL
            | STRING_LITERAL

<signed-number> ::= <sign> NUMBER
<sign> ::= "+" | "-"
```

```
<number_statement> ::= NUMBER | DOT(.) NUMBER
```

```
<signed_number> ::= "+" NUMBER | "-" NUMBER
```

```
<repeat_statement> ::= KEYWORD(ulangi) <statement-list>
                       KEYWORD(sampai) <expression>
```

- Pembagian Tugas:

No.	NIM	Persentase Kontribusi

1.	13523023	25%
2.	13523025	25%
3.	13523051	25%
4.	13523111	25%

REFERENSI

Aho, Alfred V. *Compilers: Principles, Techniques, & Tools*. Pearson/Addison Wesley, 2007, https://repository.unikom.ac.id/48769/1/Compilers%20-%20Principles,%20Techniques,%20and%20Tools%20(2006).pdf. Accessed 16 October 2025.

Hopcroft, John E., et al. *Introduction to Automata Theory, Languages, and Computation*. 3rd ed., Pearson/Addison Wesley, 2007, https://cdn-edunex.itb.ac.id/29161-Formal-Language-Theory-and-Automata/1629640939613_Introduction-to-Automata-Theory,-Languages,-and-Computation-Edition-3.pdf. Accessed 16 October 2025.

Wirth, Niklaus. *PASCAL-S. A Subset and its Implementation*. http://pascal.hansotten.com/uploads/pascals/PASCAL-S%20A%20subset%20and%20its%20Implementation%20012.pdf. Accessed 16 October 2025.